Daniel Brenner, Dinh Xuan Truong Ung, Amine Rekik, Roman Pauli

Analyseaufgabe 04 – Manipulation von Bytecode

4_1

Annahmen und Voraussetzungen:

Dateien:

- Das Projekt muss sich im Verzeichnis ./4_1/Projekt befinden.
- Die Methoden, auf die getestet werden soll, sollen in die Datei ./4_1/Input.txt geschrieben werden.
    - KlassenName_MethodenName_(Parameter)RückgabeWert
    - java/lang/System setSecurityManager (Ljava/lang/SecurityManager;)V
        - ⇨ auf Leerzeichen achten

Kommandozeilenparameter:

Der **Run-Befehl** muss wie folgt aufgebaut sein (die Reihenfolge der Parameter ist beliebig, Leerzeichen dazwischen):

- file=untitled.jar => Das zu prüfende Projekt
- cg=CHA/RTA/XTA => Der Algorithmus, der für den Callgraph verwendet werden soll.
- il=Boolean => Gibt an, ob das Projekt eine Library ist (true = ist eine Library, false = ist keine Library).

Bspw.:

file=untitled.jar[_]il=false[_]cg=XTA

Konkrete Nutzung:

Wenn sich alle Dateien an den passenden Orten befinden, kann das Projekt wie gewohnt gestartet werden. Während der Ausführung werden die Instruktionen der Methoden, die auf mindestens eine der gesuchten zugreift, ausgegeben. Hierbei werden in jeder Zeile zuerst die gefundenen Instruktionen und dann die neuen Instruktionen, die von der neu erstellten .class übernommen werden, ausgeben (meist Identisch). Die Instruktionen sind in verbesserter Form auch nachträglich einsehbar in dem ./4_1/Instructions.txt Textdokument.

Die dabei erstellten .class Dateien sind nachträglich alle im ./4_1/Output Verzeichnis zu finden.

Konfigurationsmöglichkeiten:

Es können die CallGraph-Algortihmen CHA, RTA und XTA genutzt werden, wobei wir XTA empfehlen.

Man kann sich bei erneutem Programmstart die vorher erstellten .class Dateien ausgeben lassen, indem man im Quellcode in Zeile 41 printClassFiles() nutzt.

Da die Eingabe, der zu suchenden Methoden, schnell zu ungewünschten Ergebnissen führen kann, kann man sich die ‚für jede Zeile erstellten, ComparableObjects (jeweils eine Zeile aus Input.txt) ausgeben lassen in der Methode readFile() Zeile 89.

Falls man verändern möchte, wie Instruktionen, die kritische Methoden aufrufen, ersetzt werden, muss man die Methode ModifyClasses.testForSearchedMethod verändern.

4_2

Annahmen und Voraussetzungen:

Dateien:

- Das Projekt muss sich im Verzeichnis ./4_2/Projekt befinden.
- Die Library muss sich im Verzeichnis ./4_2/Library befinden (Eingabe ist zwar nur eine, aber es können mehrere in dem Verzeichnis sein ohne Problemen zu machen).

Kommandozeilenparameter:

Der **Run-Befehl** muss wie folgt aufgebaut sein (die Reihenfolge der Parameter ist beliebig, Leerzeichen dazwischen):

- file=commons-collections4-4.5.0-M2.jar => Projekt
- cg=CHA/RTA/XTA => CallGraph-Algorithmus
- il=Boolean => Gibt an, ob das Projekt eine Library ist (true = ist eine Library, false = ist keine Library).

Bspw.:

file=commons-collections4-4.5.0-M2.jar[_]il=true[_]cg=XTA[_]dp=commons-lang3-3.14.0.jar

Konkrete Nutzung:

Wenn sich alle Dateien an den passenden Orten befinden, kann das Projekt wie gewohnt gestartet werden. Die erstellten .class Dateien befinden sich nach Ausführung in dem ./4_2/Output Verzeichnis.

Konfigurationsmöglichkeiten:

Es können die CallGraph-Algortihmen CHA, RTA und XTA genutzt werden, wobei wir XTA empfehlen.

Man kann sich bei erneutem Programmstart die vorher erstellten .class Dateien ausgeben lassen, indem man im Quellcode in Zeile 40 printClassFiles() nutzt (nicht empfohlen, weil sehr viele).